# Afar-PrgEnv: a configurable programming environment for AMD Afar compilers for Cray EX systems

Oscar Hernandez
oscar@ornl.gov
Oak Ridge National Laboratory
Oak Ridge, Tennessee, USA

Wael Elwasif
elwasif@ornl.gov
Oak Ridge National Laboratory
Oak Ridge, Tennessee, USA

## Abstract

Afar-prgenv is a module-based overlay that enables early evaluation of AMD compiler drops on Cray/HPE systems without disrupting the Cray Programming Environment. The tool integrates AFAR compiler and ROCm drops into the PE hierarchy, preserves the ‘ftn/cc/CC‘ workflow, and stabilizes MPI and library metadata through pkg-config shims and wrapper diagnostics. We describe the design and implementation of Afar-prgenv and evaluate it on Frontier using the current PE and AFAR drop series. Our evaluation uses a reproducible test matrix covering MPI, OpenMP offload, and key Cray libraries (HDF5, FFTW, LibSci, DSMML, and PnetCDF), followed by extended validation with the SOLLVE suite and an OLCF Fortran application set. The results show that Afar-prgenv makes early compiler drops immediately usable with production libraries and support tools, providing a repeatable path to certify monthly drops before general availability. This work positions Afar-prgenv as a user support tool that bridges the gap between vendor release cadence and production readiness.

**ACM Reference Format:**
Oscar Hernandez and Wael Elwasif. 2026. Afar-PrgEnv: a configurable programming environment for AMD Afar compilers for Cray EX systems. In *Proceedings of 13th International Workshop on HPC User Support Tools (SC25) (HUST ’26)*. ACM, New York, NY, USA, 10 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

## 1 Introduction

Cray/HPE systems provide a stable programming environment (PE) that couples compilers, MPI, and optimized libraries behind consistent ‘ftn/cc/CC‘ wrappers. This stability is essential for production workloads, but it slows adoption of new compiler drops from AMD, particularly when the Cray PE does not yet ship the corresponding toolchain. Support teams and early adopters are left with a choice: wait for vendor integration or accept a fragmented environment that cannot interoperate with PE libraries and tooling.

Afar-prgenv closes this gap.[4] It overlays AMD AFAR compiler and ROCm drops into the Cray PE module hierarchy while preserving the PE contract that users and build systems depend on. The overlay provides wrapper logic for OpenMP offload, MPI metadata re-synchronization, and pkg-config shims that keep library flags stable even when modules are swapped. The result is a workflow where new drops can be tested monthly without breaking production tooling, and without requiring users to abandon the PE conventions they rely on.

### 1.1 Contributions

- Define the user-support problem for early compiler evaluation on Cray/HPE systems and motivate the need for a safe, reversible overlay.
- Present the Afar-prgenv design and implementation, including its module hierarchy, wrapper behavior, and pkg-config integration.
- Evaluate Afar-prgenv on Frontier with representative compile, link, and offload workflows, and report operational outcomes for support teams.

## 2 Motivation

Frontier and similar Cray/HPE systems evolve rapidly, while vendor-supported programming environments lag behind emerging AMD compiler and ROCm drops. User support teams need a mechanism to validate new compiler stacks without disrupting production workflows or forcing users to rebuild their toolchains from scratch.

AFAR drops arrive on a roughly monthly cadence. Each drop must be validated for basic functionality, compatibility with the PE module hierarchy, and correctness in mixed environments where MPI and vendor libraries are required. Without an overlay, AFAR drops are effectively isolated: they cannot cleanly consume Cray libraries or interact with standard build systems that rely on `ftn/cc/CC`. In practice, this makes AFAR drops unusable for production users until a support tool like Afar-prgenv bridges the gap.

The support burden is amplified by module ordering and metadata coupling. Users often load library modules before the compiler stack, and ‘pkg-config‘ metadata can become stale when MPI or PE libraries are swapped after a drop is loaded. These issues are easy to miss in ad hoc testing but cause immediate failures in user builds. Afar-prgenv is motivated by the need to make drop evaluation deterministic under real-world module ordering.

### 2.1 User Support Challenges

Support teams must bridge mismatched compiler/runtime versions, conflicting modules, and inconsistent compile/link metadata across rapidly changing software stacks. Even when an AFAR drop compiles user code, it can fail at link time due to missing PE library

**Unpublished working draft. Not for distribution.**

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
*HUST ’26, Chicago, IL, USA*
© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-x-xxxx-xxxx-x/YYYY/MM
https://doi.org/10.1145/nnnnnnn.nnnnnnn



Unpublished working draft. Not for distribution.

metadata or incompatible Fortran modules. These issues are difficult to triage because the failure surface spans compiler wrappers, MPI module files, and library search paths.

## 2.2 Use Cases

- Early validation of AMD compiler drops against Cray PE workflows.
- Compatibility testing for MPI, OpenMP offload, and vendor libraries.
- Reproducible support environments for debugging user issues.

# 3 Background

This section summarizes the Cray Programming Environment, the AFAR compiler drop model, and the Frontier system context used in this study.

## 3.1 Cray Programming Environment

The Cray PE exposes consistent compiler drivers (`ftn`, `cc`, and `CC`) backed by a module hierarchy. Users load a `cpe/<ver>` module and a corresponding `PrgEnv-*` module (e.g., `PrgEnv-amd/8.6.0`) which configure compiler wrappers, MPI integration, and library metadata. The wrappers embed PE-specific assumptions about flag handling and `PKG_CONFIG_PATH` (including the `PE_AMD_FIXED_PKGCONFIG_PATH` convention), and most build systems assume those semantics remain stable. The PE module hierarchy layers target modules (e.g., `craype-x86-trento`, `craype-accel-amd-gfx90a`), MPI, and vendor libraries on top of the base compiler environment. This hierarchy is designed to keep wrapper behavior consistent even when users mix additional library modules or swap MPI implementations, which places a premium on stable metadata paths for 'pkg-config' and compiler wrappers.

## 3.2 AMD Compiler and ROCm Drops

AFAR drops bundle AMD compilers with a matched ROCm toolkit, and they are released on a rapid cadence. Each drop requires validation against Cray PE libraries and MPI, but the drops themselves do not integrate with the PE module hierarchy. Afar-prgenv provides that integration layer while preserving PE wrapper behavior.

At the time of this evaluation, the latest AFAR drop available on Frontier is 22.3.0-10004, paired with `rocm/7.0.2` and `amd/7.0.2`. While AFAR drops ship their own ROCm runtime, site workflows may still load a system ROCm module (such as 'rocm/7.0.2') to satisfy dependencies of selected vendor libraries that lag behind the drop. AFAR drops also package MPI module files under versioned include directories (`include/mpich3.4a2` or `include/mpich4.3.1`) to match Cray MPICH 8.x or 9.x layouts. Maintaining compatibility with these layouts is critical for Cray PE builds that rely on `pkg-config` to discover MPI module paths.

## 3.3 Metadata and pkg-config

Cray PE libraries expose their compile and link flags through '.pc' files consumed by 'pkg-config'. Build systems rely on this metadata to resolve library search paths and include directories. Afar-prgenv relies on 'pkg-config' to keep MPI and vendor library metadata aligned with AFAR compilers, and it uses shim '.pc' files when PE modules do not provide compatible metadata.

## 3.4 Frontier as a Target System

Frontier is an HPE Cray EX system with AMD GPUs. The system uses a module-driven PE; at the time of this work, `cpe/25.09` and `PrgEnv-amd/8.6.0` are available. The recommended baseline for AFAR testing in this workspace loads `rocm/7.0.2` and `amd/7.0.2` alongside `PrgEnv-amd`, then adds the AFAR module tree from `/sw/crusher/ums/compilers/modulefiles/`. The target accelerator for offload testing is AMD GFX90a. We select it via the `craype-accel-amd-gfx90a` module in the PE stack.

# 4 Related Work

Module systems such as Environment Modules and Lmod underpin most HPC software stacks by organizing compiler, MPI, and library families into hierarchical module trees.[2, 6] Package managers like Spack and EasyBuild automate dependency resolution and software installation across large software stacks.[1, 3] These tools are essential for building and distributing software, but they do not preserve PE-specific wrapper semantics or the MPI metadata coupling that Cray/HPE systems rely on.

Cray PE documentation emphasizes the role of 'ftn/cc/CC' wrappers and PE metadata paths in maintaining portable build workflows across modules and targets.[5] Compiler wrapper techniques are widely used to inject site defaults or architecture flags, yet they typically assume a single compiler stack rather than a layered environment that overlays experimental toolchains onto a production PE. Afar-prgenv complements these approaches by focusing on the user-support problem of early compiler evaluation in a production PE, with an emphasis on preserving the 'ftn/cc/CC' contract and vendor library integration.

Validation suites such as SOLLVE's OpenMP V&V provide systematic coverage of OpenMP semantics and offload behavior,[8, 9] and MPI implementations commonly ship their own test suites.[7] Afar-prgenv targets a different layer: the interoperability between vendor compiler drops, PE wrappers, and PE-provided libraries. Its repro suite prioritizes build and link correctness, while existing validation suites are used after the PE compatibility gate to assess functional readiness in application workflows.

Finally, site operators often maintain ad hoc module overlays to stage experimental compilers, but these overlays typically bypass the Cray PE wrapper contract or require users to hand-curate library flags. Afar-prgenv formalizes that overlay into a reproducible, metadata-driven workflow that keeps the wrapper behavior intact while exposing the new toolchain to existing PE libraries and diagnostics.

# 5 Design Goals

The design targets minimal disruption to the Cray PE workflow, reproducibility for support teams, and portability across Cray/HPE systems. The primary assumption is that user build systems and support tooling already depend on PE wrappers and module semantics.

- Preserve Cray wrapper semantics while enabling AMD compiler drops.

Unpublished working draft. Not for distribution.

- Keep module interactions deterministic and reversible.
- Provide consistent MPI and library metadata via pkg-config.
- Support rapid testing with clear diagnostics for support staff.

Beyond these goals, the design explicitly targets module-order independence and diagnostic completeness. Support teams need to reproduce user environments where libraries may be loaded before or after AFAR, so Afar-prgenv treats 'pkg-config' metadata and wrapper state as dynamic rather than static at module load time.

## 5.1 Readiness Criteria

We define readiness as the ability to compile and link common MPI, OpenMP offload, and PE library workflows using the standard 'ftn/cc/CC' wrappers. Repro tests are treated as gates: a drop is considered ready for broader validation only after the repro suite reports PASS for core workflows and explicitly documents any SKIP conditions. This gating model prioritizes correctness and compatibility before any performance tuning or application regression testing.

## 5.2 Non-Goals

We do not replace the Cray PE or provide a new package manager. Instead, we focus on the narrow but critical gap between AFAR drop availability and PE integration, prioritizing correctness and compatibility over performance tuning.

# 6 Architecture and Implementation

Afar-prgenv integrates AMD compiler drops into the Cray PE by providing a module hierarchy and lightweight wrapper logic. The overlay is intentionally minimal: it avoids replacing the PE and instead augments it with AFAR-specific paths, metadata, and diagnostics.

## 6.1 Module Hierarchy

The module tree is split into a meta module (`afar-prgenv/<ver>`) and two component modules: `afar-rocm/<ver>` and `afar-amd/<ver>`. The meta module unloads conflicting vendor modules, then loads the ROCm toolkit module followed by the compiler module to ensure the compilers are last in 'PATH'. This keeps the Cray PE wrappers stable while redirecting the underlying toolchain.

## 6.2 Compiler Wrappers

Afar-prgenv ships 'ftn', 'cc', and 'CC' wrappers that preserve Cray PE behavior while injecting AFAR-specific fixes. The wrappers:

- Resolve the real Cray compiler driver and delegate to it.
- Clear `CRAY_ACCEL_*` to avoid unsupported PE offload flags.
- Inject '–offload-arch' when '-fopenmp' is present and no offload arch is explicitly specified.
- Re-synchronize MPI metadata at invocation time to match the active 'cray-mpich' module.
- Filter or replace incompatible flags using a configurable rule set.

## 6.3 Wrapper Filtering and Link Type Guard

Wrapper filters ignore or replace arguments injected by build systems that are incompatible with AFAR. The filter is defined in a shared rule file and can be extended per site via `AFAR_WRAPPER_FILTER_EXTRA`. The wrappers also guard against Cray's dynamic link mode by unsetting `CRAYPE_LINK_TYPE=dynamic`, preventing late-injected flags that conflict with the AFAR toolchain.

**Table 1: Selected Afar-prgenv environment overrides.**

| Variable | Purpose |
|---|---|
| `AFAR_REAL_FTN` | Override real PE 'ftn' path |
| `AFAR_REAL_CC` | Override real PE 'cc' path |
| `AFAR_REAL_CXX` | Override real PE 'CC' path |
| `AFAR_FTN_OFFLOAD_ARCH` | Force Fortran offload arch |
| `AFAR_WRAPPER_FILTER` | Custom wrapper filter file |
| `AFAR_WRAPPER_FILTER_EXTRA` | Add local filter rules |
| `AFAR_PKGCONFIG_SHIM_DIR` | Extra `.pc` shim path |
| `AFAR_PKGCONFIG_SHIM_AUTO` | Auto-refresh shims toggle |
| `AFAR_LLVM_LIB_DIR` | Override LLVM runtime fallback |

**Table 2: Default wrapper filter rules used by Afar-prgenv.**

| Pattern | Rationale |
|---|---|
| `-dynamic` | Avoid PE-injected dynamic link flags that break AFAR. |
| `-ftree-vectorize` | Drop GCC-specific flag not supported by AFAR. |
| `-ffunction-sections` | Drop GCC-specific flag in PE build flags. |
| `-fdata-sections` | Drop GCC-specific flag in PE build flags. |

## 6.4 pkg-config Integration

MPI and vendor libraries depend on consistent 'pkg-config' metadata. The generator produces AFAR-specific '.pc' files for MPI and ROCm, while a shim generator captures module-provided metadata for Cray libraries that do not ship usable '.pc' files. The wrappers mirror the PE-specific metadata path into `PKG_CONFIG_PATH` to keep build systems stable under module swaps. When required ROCm 6 runtime libraries are missing, the AFAR ROCm module suppresses Cray MPICH GTL 'pkg-config' variables to avoid injecting incompatible link flags. This keeps MPI builds functional while signaling that accelerator-enabled MPI paths need follow-up validation.

Shims can be regenerated automatically when the module environment changes, so libraries can be loaded before or after `afar-prgenv` without breaking builds. The wrappers track environment changes and re-run shim generation when necessary, keeping `PKG_CONFIG_PATH` consistent for build systems.

## 6.5 MPICH Flavor Mapping

Cray MPICH 8.x uses `mpich3.4a2` and 9.x uses `mpich4.3.1`. Afar-prgenv generates per-flavor 'mpichf90.pc' files and selects the correct flavor based on `CRAY_MPICH_VERSION`. The wrappers re-check this at compile time so MPI metadata remains correct even if users swap MPICH modules after loading AFAR.

## 6.6 LLVM Runtime Fallback

Some AFAR drops omit LLVM runtime libraries. Examples include `libpgmath.so` and `libflang.so`. The module logic detects the newest compatible `/opt/rocm-*/llvm/lib` and appends it as a

Unpublished working draft. Not for distribution.

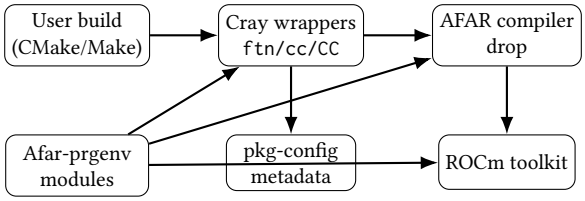


**Figure 1: Afar-prgenv overlay architecture. The Cray PE remains the primary interface; Afar-prgenv inserts AFAR compilers and ROCm while preserving PE wrapper semantics and library metadata.**

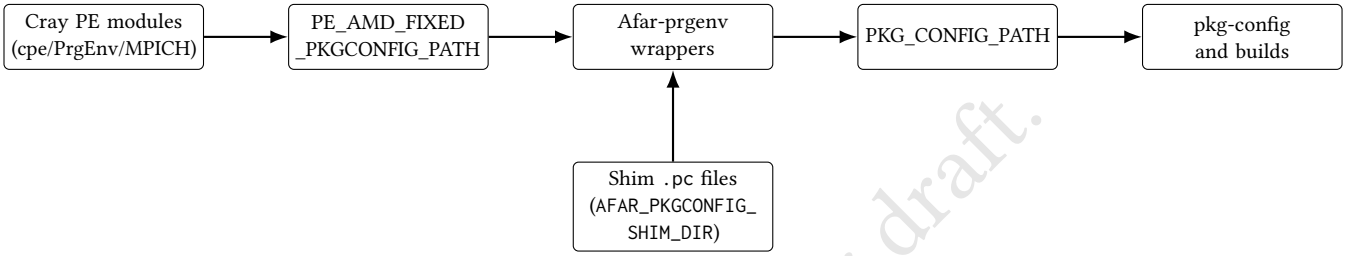


**Figure 2: pkg-config metadata flow under Afar-prgenv.**

Unpublished working draft. Not for distribution.

fallback, unless a site overrides the path with `AFAR_LLVM_LIB_DIR`. This preserves compiler functionality without modifying the upstream drop.

### 6.7 Generation and Configuration

Version mappings and PE paths are defined in configuration files and are regenerated with a single script. This keeps the module tree reproducible and aligns the AFAR drop version with the ROCm toolkit and MPI metadata used in tests. The generator also writes diagnostics that the support team uses to track drop readiness over time.

## 7 Operational Workflow

This section describes how support teams deploy Afar-prgenv and how users interact with it during early drop evaluation.

### 7.1 Support Team Workflow

Support staff add a new AFAR drop under the configured root, update the version mapping, and regenerate the module tree using `afar_modules/scripts/generate_afar_modules.sh`. They then run the repro test suite via `scripts/run_afar_tests.sh -keep-going` and capture logs for validation. A drop is considered ready for user evaluation when the repro matrix passes and the wrapper diagnostics show stable MPI and library metadata. The harness writes per-profile logs under `logs/`, which serve as support artifacts for regression tracking.

When PE library modules do not provide usable 'pkg-config' metadata, the support workflow also includes generating shim files via `generate_pkgconfig_shims.sh`. This step can be triggered explicitly or left to the wrapper auto-refresh logic, but it is recorded in logs so that metadata changes are traceable during drop triage.

After repro gating, support staff run extended validation suites, including an OLCF Fortran application set and the SOLLVE validation suite, to assess production readiness beyond basic compile and link coverage.

### 7.2 Admin Deployment

For site deployment, the AFAR module tree (`modulefiles`, `bin`, `pkgconfig`, and configuration files) is copied into a site modulepath. Admins ensure wrapper scripts are executable and add the module path via 'module use'. Updates to AFAR drops or MPICH paths are performed by refreshing the version mapping and re-running the generator, followed by a smoke test ('ftn', 'pkg-config –cflags mpichf90') to confirm wrapper and metadata health.

### 7.3 User Workflow

Users follow the standard PE module sequence (e.g., 'cpe/25.09', 'PrgEnv-amd/8.6.0') and then load 'afar-prgenv/22.3.0-10004'. The wrappers preserve the expected compile and link commands while injecting AFAR paths and offload flags when needed.

### 7.4 Failure Modes and Diagnostics

Common failure modes include missing or mismatched MPI metadata, incompatible Fortran module files from vendor libraries, and absent ROCm runtime libraries. Wrapper diagnostics surface these issues early by reporting filtered flags, active MPI flavors, and updated `PKG_CONFIG_PATH` settings.

The repro harness also exercises different module load orders to ensure that the wrappers re-synchronize MPI and library metadata correctly when users load modules before or after 'afar-prgenv'.

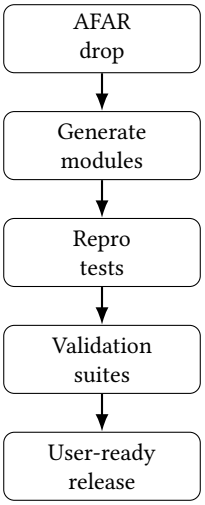


**Figure 3: Monthly AFAR drop workflow. Afar-prgenv provides a repeatable path from drop ingestion to user readiness.**

Unpublished working draft. Not for distribution.

## 8 Repro Harness and Test Inventory

This section details the repro harness used to gate AFAR drops and the catalog of tests that exercise PE wrapper, MPI, OpenMP, and library integration.

### 8.1 Harness Architecture

The repro harness is driven by `scripts/run_afar_tests.sh`. It defines named profiles that extend a baseline PE stack with a specific library module and then runs every repro under that profile. By default, the harness loads each library module both before and after `afar-prgenv` to validate module-order independence. The ordering can be restricted with `-lib-order before|after|both` when a focused regression is needed.

Profiles can be overridden by environment variables so support teams can validate alternate PE versions or modulepaths without editing the scripts. Common overrides include `AFAR_TEST_CPE`, `AFAR_TEST_MPICH`, and `AFAR_TEST_MODULEPATH`. The harness also supports targeted execution through `-profiles` and `-keep-going`, enabling incremental validation when a drop is known to have a single regression.

### 8.2 Repro Case Catalog

The catalog emphasizes compatibility with PE metadata rather than numerical performance. Every repro compiles and links via 'ftn/cc/CC' to ensure wrapper semantics remain intact across profiles.

### 8.3 Logging and Diagnostics

Log files are written under `logs/` and include a timestamp plus profile suffix (for example, `20240320-base.log`); the location can be overridden with `AFAR_TEST_LOG_DIR`. These logs capture wrapper warnings, 'pkg-config' paths, and linker diagnostics that support teams use to triage regressions. For deeper debugging, the wrappers support `AFAR_WRAPPER_DEBUG` to emit the final argument list passed to the underlying Cray PE drivers.

## 9 Experimental Setup

We evaluate Afar-prgenv on Frontier using the current Cray PE and the latest AFAR drop available in the system module tree. The goal is to measure compatibility with the PE toolchain and key libraries, not peak performance.

### 9.1 Systems and Software

Our baseline environment loads 'cpe/25.09' and 'PrgEnv-amd/8.6.0' alongside 'rocm/7.0.2' and 'amd/7.0.2'. Afar-prgenv modules are provided from '/sw/crusher/ums/compilers/modulefiles/', and the evaluation targets 'afar-prgenv/22.3.0-10004'. This setup mirrors the site-recommended module sequence used by the support team.

Although the AFAR drop ships a ROCm runtime, the additional 'rocm/7.0.2' module is retained for compatibility with PE libraries that still depend on system ROCm paths. When evaluating older dependencies, we also verify compatibility against ROCm 6.x where needed. The repro harness focuses on PE and AFAR modules; ROCm compatibility for legacy dependencies is tracked as a separate gate (e.g., 'libsci-acc').

```
module purge
module reset
module load cpe/\peversion
module load rocm/\afarrocm
module load amd/\afarrocm
module load PrgEnv-amd
module load cray-mpich/9.0.1
module load craype-x86-trento
module load craype-accel-amd-gfx90a
module use /sw/crusher/ums/compilers/modulefiles/
module load afar-prgenv/\afarversion
```

### 9.2 Workloads and Repro Cases

We use the repository repro suite to validate compile and link correctness: 'compile-only', 'cmake-mpi-omp', 'wrapper-compile-link', 'mpi-hello', 'omp-c', 'omp-f90', 'libsci', 'libsci-acc', 'hdf5', 'fftw',

**Table 3: Repro cases and their validation focus.**

| Repro | Focus | Requirements/Notes |
|---|---|---|
| compile-only | Fortran compiler sanity | AFAR 'ftn'; no link step. |
| cmake-mpi-omp | CMake wrapper pass-through | CMake in `PATH`; uses 'ftn' and 'cc'. |
| wrapper-compile-link | Wrapper compile/link split | Uses `scripts/check_ldd.sh` on outputs. |
| mpi-hello | MPI module integration | Requires `cray-mpich`; `AFAR_TEST_RUN_MPI` enables runtime. |
| omp-c | OpenMP offload (C) | Requires `craype-accel-amd-gfx90a` or `AFAR_CC_OFFLOAD_ARCH`. |
| omp-f90 | OpenMP offload (Fortran) | Requires `craype-accel-amd-gfx90a` or `AFAR_FTN_OFFLOAD_ARCH`. |
| libsci | LibSci links | Uses `pkg-config libsci`. |
| libsci-acc | LibSci accelerator links | Uses `AFAR_LIBSCI_ACC_LIBNAME`; skipped if ROCm 6 deps missing. |
| hdf5 | HDF5 serial/parallel | Uses AFAR-compiled Fortran shim; requires 'pkg-config hdf5'. |
| fftw | FFTW bindings | Uses `pkg-config fftw3`. |
| dsmml | DSMML bindings | Uses `pkg-config dsmml`. |
| netcdf | PnetCDF bindings | Uses 'pnetcdf.inc'; skips Fortran if include is missing. |

**Table 4: Default repro profiles and module extensions.**

| Profile | Module(s) | Purpose |
|---|---|---|
| base | — | compiler/MPI/offload sanity |
| libsci | cray-libsci | BLAS/LAPACK links |
| libsci-acc | cray-libsci_acc | accelerator libs |
| hdf5 | cray-hdf5-parallel | HDF5 parallel stack |
| hdf5-serial | cray-hdf5 | HDF5 serial stack |
| fftw | cray-fftw | FFTW bindings |
| dsmml | cray-dsmml | DSMML links |
| netcdf | cray-parallel-netcdf | PnetCDF stack |

'dsmml', and 'netcdf'. The tests exercise the PE wrappers, MPI module files, and library metadata via `pkg-config`, and `scripts/check_ldd.sh` validates the resulting binaries. The CMake repro requires `cmake` in `PATH`.

The repro harness (`scripts/run_afar_tests.sh`) runs profiles that load each library module both before and after 'afar-prgenv', ensuring the wrapper re-synchronization logic works under module-order permutations.

In this study we ran:

```
scripts/run_afar_tests.sh --keep-going \
  --afar-version \afarversion \
  --afar-module afar-prgenv/\afarversion
```

The default profiles exercised by the harness are summarized in Table 4. Each profile extends the baseline module stack with a library module and its associated 'pkg-config' metadata, enabling focused validation of PE libraries under AFAR.

The harness can restrict module ordering with `-lib-order` and can limit profiles with `-profiles`. Environment overrides such as `AFAR_TEST_CPE` and `AFAR_TEST_MPICH` enable alternate PE combinations, while `AFAR_TEST_RUN` or `AFAR_TEST_RUN_MPI` trigger runtime execution when nodes are available. These controls let support teams focus on regression hotspots without reworking individual repro scripts.

## 9.3 Metrics

The primary metric is functional readiness: all repro cases must compile and link with the PE wrappers and AFAR drop. Secondary metrics include the diagnostic completeness of wrapper logs, P/S/F rates per test, and the time to validate a new drop. We also track module-order robustness by running the repro suite under multiple module load permutations to ensure metadata re-synchronization remains correct.

**Table 5: Repro suite coverage for the AFAR drop 22.3.0-10004. Status is recorded as P/S/F for each test.**

| Test | Coverage | Status (P/S/F) |
|---|---|---|
| cmake-mpi-omp | CMake MPI+OMP | PASS |
| compile-only | compiler sanity | PASS |
| wrapper-compile-link | wrapper compile/link | PASS |
| mpi-hello | MPI module integration | PASS |
| omp-c | OpenMP offload (C) | PASS |
| omp-f90 | OpenMP offload (F) | PASS |
| libsci | PE math libs | PASS |
| libsci-acc | accelerator libs | SKIP |
| hdf5-parallel | HDF5 (par) | SKIP |
| hdf5-serial | HDF5 (ser) | PASS |
| fftw | FFTW (C/C++/F) | PASS |
| dsmml | DSMML | PASS |
| netcdf | PnetCDF | PASS |

## 10 Evaluation and Results

We evaluate Afar-prgenv against the functional readiness criteria defined in Section 9.3. The emphasis is on correctness and compatibility with PE libraries and MPI, which are prerequisites for any further performance or application-level validation.

The 'libsci-acc' profile is intentionally skipped; `libsci_acc` linking is not yet validated and may depend on ROCm 6 runtime libraries. The parallel HDF5 test is skipped due to missing HDF5 C++ 'pkg-config' metadata, while the serial HDF5 path passes.

Across the repro matrix, 11 of 13 tests pass, 2 are skipped, and none fail. Under our readiness policy, this qualifies the drop for extended validation because the skips correspond to known external dependencies rather than compiler regressions.

## 10.1 Compilation and Linking

The repro matrix validates that AFAR drops compile and link with the PE wrappers without requiring manual flag surgery. Wrapper diagnostics report any filtered flags or mismatched MPI metadata early, which reduces support turnaround when a drop regresses. All workflow-focused repros ('compile-only', 'cmake-mpi-omp', and

Unpublished working draft. Not for distribution.

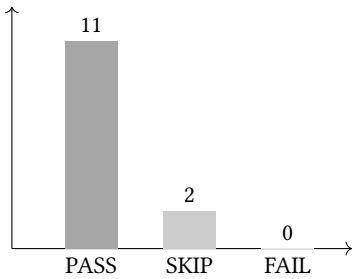


**Figure 4: Repro suite outcomes for 22.3.0-10004(unique tests).**

**Table 6: Skip causes observed in the 22.3.0-10004repro run.**

| Repro | SKIP cause |
|---|---|
| libsci-acc | ROCm 6 runtime dependencies missing for accelerator links. |
| hdf5-parallel | Missing HDF5 C++ pkg-config metadata for the PE module. |

**Table 7: Module ordering permutations used to validate metadata re-synchronization.**

| Permutation | Example | Status (P/S/F) |
|---|---|---|
| Libs before AFAR | load libs, then afar-prgenv | PASS |
| Libs after AFAR | load afar-prgenv, then libs | PASS |
| MPI swap after AFAR | swap cray-mpich post-load | N/A |

'wrapper-compile-link') pass for 22.3.0-10004. The CMake repro confirms that build systems can consume the wrappers without manual flag translation.

## 10.2 Repro Harness Coverage

The harness runs each profile under multiple module-order permutations, which stress-test the wrapper logic that re-synchronizes 'pkg-config' metadata at compile time. This ensures that users can load library modules either before or after 'afar-prgenv' without breaking builds. In this evaluation, the base profile and all library profiles completed without failures; the only exceptions were the intentional skips noted in Table 5.

## 10.3 Module Ordering Robustness

To reflect real user behavior, we execute the repro suite across multiple module load permutations (e.g., library modules loaded before or after 'afar-prgenv'). This checks that wrapper logic correctly re-synchronizes MPI and library metadata regardless of module ordering. We report P/S/F outcomes per permutation and flag any regressions for follow-up.

The before/after permutations pass for the library profiles exercised in this study (LibSci, FFTW, DSMML, PnetCDF, and serial HDF5). MPI swap after AFAR is a planned check and was not exercised in this run.

## 10.4 MPI and Offload Validation

MPI validation confirms that the wrappers resynchronize the MPI flavor on each invocation, preventing stale 'pkg-config' metadata when modules are swapped. OpenMP offload tests confirm that '–offload-arch' is injected for both C and Fortran workflows. This is exercised with the PE module `craype-accel-amd-gfx90a`. Overrides via `AFAR__OFFLOAD_ARCH` are also covered. The 'mpi-hello', 'omp-c', and 'omp-f90' repros all pass in this evaluation.

## 10.5 Library Compatibility

Library repro tests confirm that AFAR drops can link against PE-provided HDF5, FFTW, LibSci, DSMML, and PnetCDF libraries when the correct 'pkg-config' metadata is present. The 'libsci-acc' path is treated as a conditional gate because ROCm runtime dependencies can lag behind AFAR drops. When the expected ROCm runtime libraries are present, 'libsci-acc' builds successfully; otherwise the test is skipped and the drop is held for follow-up. The 'libsci-acc' repro uses `AFAR_LIBSCI_ACC_LIBNAME` to target the correct accelerator library variant.

HDF5 validation uses an AFAR-compiled Fortran shim module to avoid incompatible vendor `.mod` files while still linking against the HDF5 C libraries. The PnetCDF repro relies on 'pnetcdf.inc' instead of compiler-specific `.mod` files, and it skips Fortran if the include file is absent. FFTW and DSMML validation use 'pkg-config' metadata and `scripts/check_ldd.sh` to ensure the link paths resolve to the expected PE libraries.

## 10.6 Library Case Studies

HDF5 highlights a common Fortran compatibility issue: vendor `.mod` files are compiler-specific and often incompatible with AFAR. The repro uses an AFAR-compiled shim module (`hdf5_shim.f90`) so the Fortran build can consume the PE-provided HDF5 C libraries without ingesting incompatible `.mod` files. This pattern generalizes to other libraries that ship Fortran modules.

LibSci-acc highlights runtime dependency drift. Cray accelerator libraries may link against ROCm 6 runtimes even when AFAR drops ship ROCm 7 toolchains. The repro detects missing ROCm 6 libraries and reports a SKIP rather than a hard FAIL, signaling a dependency gap rather than a compiler regression.

PnetCDF highlights module metadata gaps. The repro uses the Fortran include `pnetcdf.inc` instead of compiler-specific module files and skips Fortran validation when the include is missing. This keeps the C/C++ checks active while isolating Fortran metadata gaps for follow-up.

## 10.7 Binary Dependency Audits

Most repros run `scripts/check_ldd.sh` after linking to verify that library paths resolve to expected prefixes. The default allowlist includes PE paths under `/opt/cray`, AFAR roots, and standard system locations (e.g., `/lib*`, `/usr/lib*`), plus ROCm runtime paths required by MPI. This audit catches accidental leakage to incompatible system libraries and provides a fast signal for regressions introduced by a new drop.

Unpublished working draft. Not for distribution.

**Table 8: Common failure signals and mitigations.**

| Signal | Mitigation |
|---|---|
| Missing 'pkg-config' metadata | Regenerate shim '.pc' files and verify `PKG_CONFIG_PATH`. |
| Wrong MPICH flavor selected | Rebuild via wrappers or reload `afar-prgenv` to resync `CRAY_MPICH_VERSION`. |
| OpenMP offload arch missing | Load `craype-accel-amd-gfx90a` or set `AFAR_FTN_OFFLOAD_ARCH`. |
| LibSci-acc link fails | Check ROCm runtime deps or defer until matching accelerator libs are available. |
| Incompatible Fortran `.mod` | Build AFAR-compatible shim modules. |

## 10.8 Metadata and Shim Coverage

Shim `.pc` files are generated per profile when PE libraries do not expose AFAR-compatible metadata. The wrappers append the shim directory to `PKG_CONFIG_PATH` and optionally refresh it when the module environment changes, which keeps library metadata stable even when users load modules out of order. In practice, this is most important for Fortran-facing libraries whose vendor '.pc' files omit module-specific include paths.

## 10.9 Diagnostic Signals

The evaluation also tracks common failure signatures that indicate a drop is not ready for broader validation. Examples include missing 'pkg-config' data for a loaded library module, a mismatch between `CRAY_MPICH_VERSION` and the selected MPI flavor, and absent ROCm runtime dependencies for accelerator libraries. The wrapper warnings and the repro harness logs capture these cases early, enabling support teams to decide whether to regenerate metadata, adjust module ordering, or defer the drop until dependencies align.

## 10.10 Operational Impact

Operationally, Afar-prgenv converts AFAR drops from isolated toolchains into PE-compatible environments. Support teams can validate a drop with a deterministic repro suite and only escalate to broader validation (Fortran test suites, OpenMP validation, and application builds) once the PE compatibility baseline is satisfied. In this run, the repro suite reported no failures; the only skips were intentional gates for 'libsci-acc' and the missing HDF5 C++ 'pkg-config' data in the parallel HDF5 path.

## 10.11 Extended Validation

After repro gating, we run an OLCF Fortran application set and the SOLLVE validation suite to evaluate readiness for production use. These suites provide broader coverage of Fortran runtime behavior, OpenMP offload correctness, and application build workflows that are not captured by the repro tests alone.

# 11 Readiness Reporting and Operations

Afar-prgenv turns monthly AFAR drops into auditable support artifacts. The repro harness emits per-profile logs, and the P/S/F outcomes are summarized in readiness reports that track drop stability across time. This operational view is important for user support because it distinguishes regressions introduced by new drops from persistent gaps in the PE ecosystem.

**Table 9: Extended validation suites used after repro gating.**

| Suite | Coverage | Status (P/S/F) |
|---|---|---|
| OLCF Fortran apps | Fortran build/runtime | TBD |
| SOLLVE V&V | OpenMP offload | TBD |

**Table 10: Readiness labels and operational response.**

| Label | Operational response |
|---|---|
| PASS | Proceed to extended validation and user-facing evaluation. |
| SKIP | Document precondition, track dependency, rerun when resolved. |
| FAIL | Block release; isolate regression and update wrapper or metadata. |

Readiness reports capture the AFAR drop ID, PE version, ROCm version, and a summary of PASS/SKIP/FAIL counts, plus the explicit reason for each SKIP. This keeps dependency gaps visible (for example, missing ROCm 6 runtime libraries or incomplete 'pkg-config' metadata) while allowing early adopters to experiment with drops that otherwise meet functional readiness criteria.

## 11.1 Readiness Labels

Each repro outcome is classified as PASS, SKIP, or FAIL. PASS indicates that the workflow compiles and links with the PE wrappers and required libraries. SKIP indicates a known precondition is missing (for example, a runtime dependency or missing metadata) and is treated as a gate rather than a regression. FAIL indicates a functional break in the drop or wrapper layer and blocks progression to extended validation.

## 11.2 Cadence and Regression Tracking

AFAR drops arrive on a rapid cadence, so readiness reporting emphasizes repeatability. The generator logs, repro outputs, and module versions are captured together so that support teams can compare a new drop against the last known-good baseline. This makes it easier to attribute regressions to a change in compiler behavior versus a change in PE library metadata.

Because the reports are structured, we can trend the pass rate across drops and flag persistent SKIP causes that should be addressed by PE library updates or wrapper fixes. Over time, this reduces the time to certify each monthly drop and clarifies which regressions are actionable by the support team.

## 11.3 From Repro Gating to User Rollout

Once the repro suite passes and known SKIP conditions are documented, support teams can stage the drop for early adopters. Additional validation suites (SOLLVE V&V and the OLCF Fortran applications) are used to confirm readiness for broader user adoption, but the repro gate remains the minimum requirement for publishing a drop in the site module tree.

Unpublished working draft. Not for distribution.

**Table 11: Primary artifacts produced or consumed by Afar-prgenv.**

| Artifact | Purpose |
|---|---|
| `afar_modules/config/` | AFAR root and version mappings |
| `afar_modules/modulefiles/` | Generated Lmod module tree |
| `afar_modules/pkgconfig/` | Generated MPI/ROCm '.pc' metadata |
| `afar_modules/bin/` | Wrapper scripts ('ftn/cc/CC') |
| `scripts/run_afar_tests.sh` | Repro harness entry point |
| `logs/` | Per-profile validation logs |

## 12 Reproducibility and Artifacts

The Afar-prgenv workflow is designed to be reproducible within a site-managed module tree. The generator script and configuration files capture the AFAR root path, version mappings, and MPICH paths used during evaluation, while the repro harness records the module stack and wrapper diagnostics for each profile.

To reproduce a drop evaluation, a site admin installs the AFAR drop under the configured root, updates the version mapping, regenerates modulefiles, and runs the repro harness. Because the modulefiles and 'pkg-config' metadata are generated from the same configuration inputs, the resulting environment is portable across Cray/HPE systems that share the PE hierarchy.

The logs capture the active module list (`module -t list`), wrapper diagnostics, and the resolved `PKG_CONFIG_PATH` settings so that metadata changes can be traced to specific module stacks. This makes the evaluation process repeatable without relying on ad hoc documentation.

The repository layout mirrors the operational workflow: configuration drives generation, generation drives module behavior, and the repro harness validates that the generated environment matches PE expectations. This separation makes it possible to compare drops using consistent inputs and to share readiness artifacts across support teams.

## 13 Discussion

We discuss the trade-offs of the overlay approach, portability to other Cray/HPE systems, and limitations observed during validation.

### 13.1 Limitations

The overlay depends on PE wrappers and module hierarchy conventions; changes in those interfaces require updates to Afar-prgenv. Some vendor libraries ship Fortran modules that are not compatible with AFAR compilers and require shims or local builds. Accelerator libraries (e.g., 'libsci-acc') may also lag when runtime dependencies are tied to older ROCm versions. Parallel HDF5 validation can also be constrained by missing C++ 'pkg-config' metadata in the PE environment, which must be addressed before full coverage.

We also intentionally avoid performance benchmarking in this work. Our goal is functional readiness and compatibility; performance evaluation is deferred until after a drop passes the compatibility gate.

### 13.2 Operational Scalability

The combination of a generator-driven module tree and a repro harness makes monthly AFAR drops tractable for support teams. Most updates are data-driven: adding a version mapping, regenerating modulefiles, and running the repro matrix. This workflow scales across multiple drops because the wrapper logic and 'pkg-config' shims are shared, while version-specific differences are captured in configuration files and module metadata.

### 13.3 Portability

Afar-prgenv generalizes to other Cray/HPE systems because it leverages the same PE module hierarchy and wrappers. The system-specific configuration is limited to the AFAR root path, version mappings, and the Cray MPICH install path, all of which are captured in configuration files.

Portability is strongest when the target system follows standard Cray PE conventions for 'ftn/cc/CC' and 'pkg-config'. Sites with non-standard module layouts may need additional wrapper filter rules or shim metadata to preserve order independence.

### 13.4 Lessons Learned

Two operational lessons stand out. First, early drop readiness is mostly a metadata problem: wrapper semantics and 'pkg-config' paths must be correct before any application testing can succeed. Second, reproducible repro tests are critical for support teams because they isolate regressions to specific libraries or wrapper behaviors. These lessons inform how we prioritize future automation and validation.

## 14 Conclusion and Future Work

Afar-prgenv enables early evaluation of AMD compiler drops on Cray/HPE systems by preserving the PE workflow while integrating AFAR compilers and ROCm toolkits. The overlay converts isolated drops into PE-compatible environments that can be validated with reproducible repro tests, giving support teams a repeatable path to certify monthly drops before broader application testing.

Future work includes expanding the validation suite to cover additional Fortran and OpenMP test suites, integrating application-level regression tests, and automating readiness reports for new drops. We also plan to formalize MPI swap-after-load tests, track ROCm runtime compatibility for accelerator libraries, and integrate the repro harness with site CI so that new drops are evaluated immediately after delivery.

## Acknowledgments

Please add

## References

[1] EasyBuild Team. 2012. EasyBuild: Building Software on HPC Systems. https://easybuild.io/. Accessed 2024-10-01.
[2] Environment Modules Project. 2024. Environment Modules. https://modules.readthedocs.io/. Accessed 2024-10-01.
[3] Todd Gamblin, Matthew LeGendre, Michael Collette, Gregory Lee, Adam Moody, Bronis de Supinski, and Scott Futral. 2015. The Spack Package Manager: Bringing Order to HPC Software Chaos. In *Proceedings of SC15*.
[4] Oscar Hernandez and Wael Elwasif. 2025. Afar-prgenv: A configurable programming environment for AMD AFAR drops on Cray HPE systems. Internal tool used at OLCF Frontier.

Unpublished working draft. Not for distribution.

[5] HPE Cray. 2024. HPE Cray Programming Environment User Guide. https://support.hpe.com/. Accessed 2024-10-01.
[6] Robert McLay, Karl Schulz, and William Barth. 2011. Lmod: A Modern Environment Module System. https://lmod.readthedocs.io/. Accessed 2024-10-01.
[7] MPICH Project. 2023. MPICH Documentation. https://www.mpich.org/. Accessed 2024-10-01.
[8] OpenMP Architecture Review Board. 2024. OpenMP Application Programming Interface. https://www.openmp.org/specifications/. Version 5.x, accessed 2024-10-01.
[9] SOLLVE V and V Team. 2020. SOLLVE OpenMP Validation and Verification Suite. https://github.com/SOLLVE/sollve_vv. Accessed 2024-10-01.

Unpublished working draft.
Not for distribution.